Emotion Detection & Learning Support Engine

Date	15 July 2026
Team ID	XXXX
Project Name	Emotion Detection & Learning Support Engine
Maximum Marks	5 Marks

Project Description:

The **AI-Driven Emotion Detection & Personalized Learning Support System** is an intelligent platform designed to analyze a learner's emotional state from their text input and provide personalized academic and emotional support. The system uses Natural Language Processing (NLP) and two independently trained deep learning models - **BiLSTM** (TensorFlow/Keras) and a fine-tuned **BERT** (PyTorch + HuggingFace Transformers) - to classify emotions across five classes: **Bored, Confident, Confused, Curious, and Frustrated**. It also includes a rule-based keyword-boosting layer to improve prediction accuracy on ambiguous input.

Once a user submits a learning-related query, the system processes the text, detects the underlying emotion (including mixed emotions such as "Bored + Frustrated"), and generates a supportive, context-aware response. A generative AI module (Gemini API) is used to provide personalized guidance and learning suggestions based on the detected emotion, with an automatic template-based fallback if the API is unavailable. The main goal of this project is to create an intelligent learning assistant that understands student emotions and helps improve their learning experience by providing motivation, guidance, and stress-free support.

The platform is an end-to-end system that transforms a learner's free-text description of their study challenge into an emotion-aware, supportive response. By combining deep learning emotion classifiers (BiLSTM and BERT) with rule-based keyword enhancement and Gemini-generated guidance, the platform ingests raw student input, predicts nuanced emotional states, and delivers tailored tips, next steps, and encouragement. This automated workflow reduces manual triage, surfaces real-time emotional insight, and maintains consistent, empathetic responses within seconds. Beyond prediction, the system logs interactions to CSV for continuous learning, visualizes emotion trends through an analytics dashboard, and supports side-by-side model comparisons. Users run the Streamlit app to enter their problem context, see mixed-emotion breakdowns, toggle AI responses, and review personalized strategies.

Scenarios

Scenario 1: Educators, TAs & Academic Support Teams

Instructors and academic support staff often spend significant time interpreting student sentiments from messages and emails before offering help. Manual reading, guessing emotions, and crafting the right tone slows response cycles and can miss struggling students. The AI Learning Assistant removes these bottlenecks by automatically classifying emotions from student text, highlighting confidence levels, and generating supportive, field-specific replies. A TA, for instance, can paste a student's concern and instantly see whether they're Confused or Frustrated, along with suggested guidance. Dual-model comparison (BiLSTM vs BERT) and mixed-emotion detection improve reliability and transparency. Teams accelerate triage, maintain consistent empathy, and focus more on targeted academic intervention rather than manual sentiment interpretation.

Scenario 2: Learners & Self-Study

Students working independently often need quick, empathetic guidance when stuck or unsure. Manually searching forums or drafting help requests takes time and may not capture their feelings. The Learning Assistant streamlines this by letting students describe their challenge in plain language and immediately receiving emotion-aware tips and next steps. For example, a learner can note "I'm lost on recursion," and the system will flag Confused, surface clarity-focused advice, and provide encouraging direction. Mixed-emotion outputs (e.g., Curious + Confused) help learners self-reflect, while saved interactions build a history for progress tracking. This makes the platform a powerful companion for self-paced study and rapid clarification.

Technical Architecture:

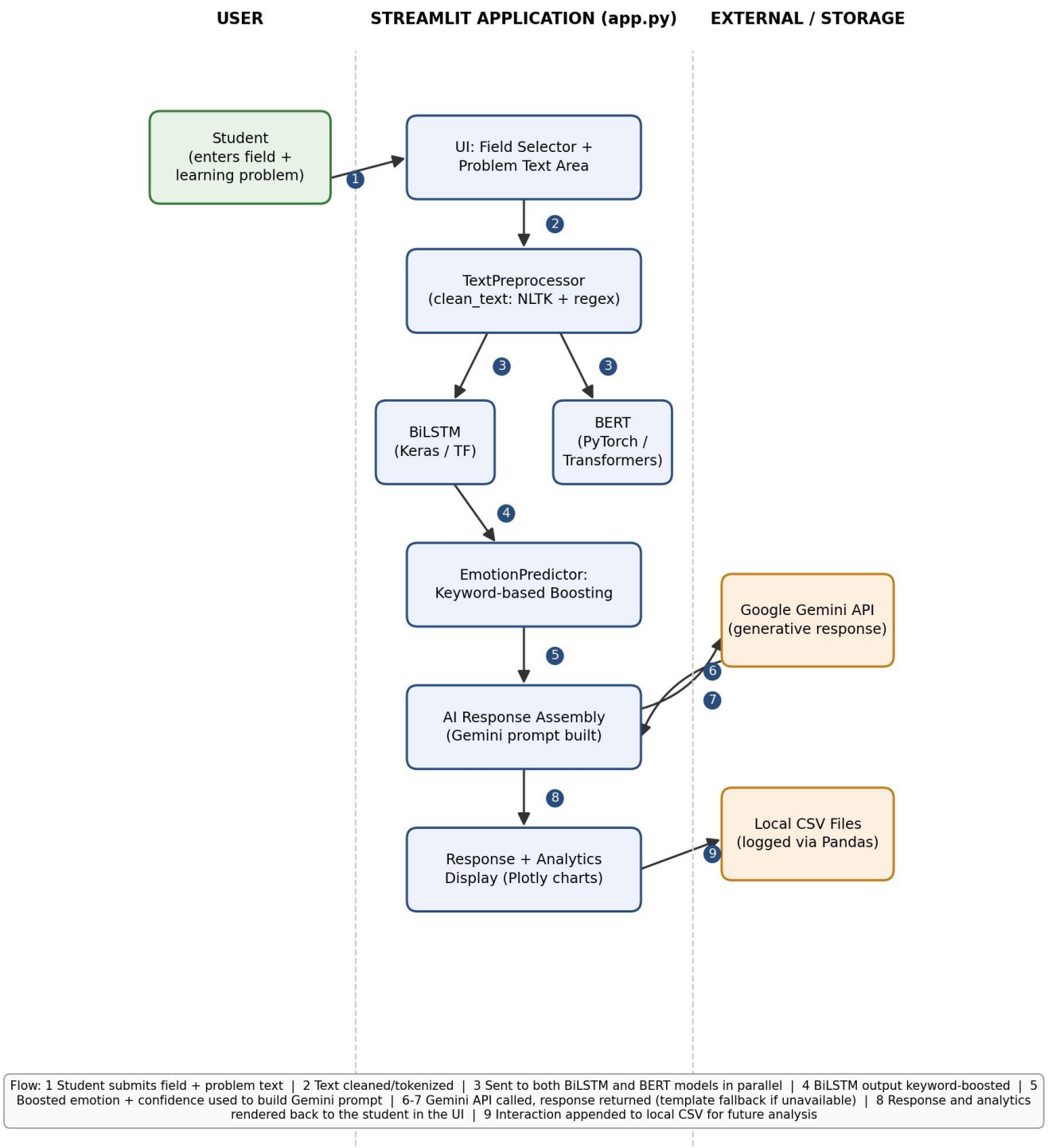

Figure 1: Architecture and data flow of the Emotion Detection and Learning Support Engine

Description: The system follows a layered architecture consisting of a Streamlit-based frontend and application layer (there is no separate backend framework such as Flask or FastAPI - all logic runs inside the Streamlit process itself), a machine learning processing layer, and a file-based storage layer. The user submits text input through the Streamlit UI, which routes it to a preprocessing module (regex + NLTK) where the text is cleaned and tokenized. The processed input is passed in parallel to two independent classifiers - a BiLSTM (Keras) and a fine-tuned BERT (PyTorch) - which each predict the user's emotional state across five classes: Bored, Confident, Confused, Curious, Frustrated. A keyword-based boosting layer then sharpens the BiLSTM's output for ambiguous cases. The detected emotion and confidence score are passed to the Gemini API to generate a personalized learning support response, with a template-based fallback if the API is unavailable. Finally, the system displays both the detected emotion and the supportive response in the UI, while logging the full interaction to local CSV files for future analysis.

Data Structure:

This application does not use a login system or a relational database, so no ER diagram of tables like Users/Login applies here. Instead, every interaction is appended as a flat row to two local CSV files, managed via Pandas. The schema of each is shown below.

emotion_response_examples.csv

Column	Description
text	The student's original problem/challenge text
emotion	Detected primary emotion (lowercased)
confidence	Model confidence score for the primary emotion (0-1)
response	The AI-generated or template-based response shown to the student
field	The student's selected academic field (e.g. Computer Science)
timestamp	ISO-format timestamp of the interaction

emotion_response_mapping.csv

Column	Description
emotion	A detected emotion label
response	The first response generated for that emotion (one row per unique emotion label)

Pre-requisites:

1. Python Environment Setup

Install Python 3.9+ and create a dedicated virtual environment for clean dependency management. Official Download: https://www.python.org/downloads/

2. Install Required Libraries

All dependencies must be installed from requirements.txt, covering: emotion modeling (TensorFlow/Keras for BiLSTM, PyTorch + HuggingFace Transformers for BERT), NLP preprocessing (NLTK), data handling and analytics (Pandas, NumPy, scikit-learn, Plotly), and the web UI plus AI integration (Streamlit, google-genai).

3. Streamlit Installation

Needed to run the interactive learning assistant UI. Docs: https://docs.streamlit.io/library/get-started/installation

4. Model Assets and Data

Ensure model files exist under models/bltsm/ (BiLSTM model, tokenizer, label encoder) and models/bert_emotion_model_final/ (BERT weights, tokenizer, config). Datasets used for training sit outside this repository (see Dataset Link in the Appendix); no extra conversion tools are required to run the app itself.

5. Gemini API Key

Required for AI-generated supportive responses. Set **GEMINI_API_KEY** in your local .env file (this is the variable name app.py actually reads via python-dotenv - not GOOGLE_API_KEY). Gemini API Setup: https://aistudio.google.com/

6. Optional Tools for Development

Recommended IDEs for development and debugging:

• Visual Studio Code: https://code.visualstudio.com/

• PyCharm Community: https://www.jetbrains.com/pycharm/

These tools support linting, Python virtual environments, and structured project navigation.

Project Workflow:

Epic 1: Environment Setup and Dependency Configuration

Focuses on setting up the complete development environment required for the Emotion Detection and Learning Support system: installing Python, configuring a virtual environment, setting up project dependencies, and integrating external services like the Gemini API.

- Story 1: Obtain Gemini API Key from Google AI Studio and store it as GEMINI_API_KEY in a local .env file.
- Story 2: Install Python 3.9+ and create/activate a virtual environment.
- Story 3: Install project dependencies from requirements.txt.
- Story 4: Create the .env configuration file with the Gemini API key.
- Story 5: Verify model directories (models/bltsm/, models/bert_emotion_model_final/) are present.
- Story 6: Prepare the overall project folder structure (src/, models/, app.py, requirements.txt).

Epic 2: Kaggle Model Training and Integration

Covers preparing, training, and integrating the BiLSTM and BERT emotion detection models, including data preprocessing, tokenization, fine-tuning, performance evaluation, and exporting trained models for use in the application.

- Story 1: Kaggle GPU environment setup with pinned dependency versions (TensorFlow 2.17.0, NumPy 1.26.4, Transformers 4.35.2, Torch 2.1.2) and validation of source datasets (GoEmotions, ISEAR, Empathetic Dialogues, plus synthetic examples for underrepresented classes).
- Story 2: Data preprocessing and tokenization - unified dataset creation combining multiple sources into a single dataframe mapped to the five target emotion classes (Bored, Confident, Confused, Curious, Frustrated); text cleaning via regex while preserving emotion-carrying punctuation; a Keras Tokenizer with a 30,000-word vocabulary, with sequences padded/truncated to 80 tokens.
- Story 3: BiLSTM model training - a Bidirectional LSTM network with an embedding layer and focal loss to handle class imbalance across the five emotion classes.
- Story 4: Domain-adaptive fine-tuning of the baseline BiLSTM on student-specific text samples, exported as bilstm_student_adaptive.keras.
- Story 5: BERT model fine-tuning - fine-tuned bert-base-uncased using the AdamW optimizer, exporting the full HuggingFace suite (safetensors, tokenizer.json, config.json).
- Story 6: Model export and local integration - migrating cloud-trained artifacts into the local models/ directory, verifying tokenizer and label-encoder metadata match the training environment exactly before Streamlit loads them.

Epic 3: Core Emotion Detection Pipeline Development

Builds the actual inference pipeline that turns cleaned text into an emotion prediction.

- Story 1: Text preprocessing and keyword-list definition (src/preprocessing.py, src/predict.py) - regex-based cleaning that preserves emotion-carrying punctuation, plus curated keyword lists per emotion class.

• Story 2: BiLSTM classifier - loads the trained model and tokenizer, pads sequences to 80 tokens, and applies softmax to produce a 5-class probability distribution.

• Story 3: BERT classifier with keyword-based adjustment - loads the fine-tuned BERT model and applies confidence/confusion keyword checks to adjust the softmax output for ambiguous cases.

• Story 4: Mixed-emotion detection - identifies secondary emotions crossing a 15% confidence threshold so more than one emotion can be surfaced (e.g. "Bored + Frustrated") instead of a single forced label.

• Story 5: Unified prediction schema - a standardized result format (emotion, confidence, all class scores) used consistently across both BiLSTM and BERT outputs.

• Story 6: CSV persistence and cached model loading - interactions are saved to CSV via Pandas, and both models are loaded once per session via Streamlit's @st.cache_resource.

Epic 4: AI-Powered Guidance and Response Generation

Integrates Gemini to turn a detected emotion into a genuinely useful, personalized response.

• Story 1: Capture the student's field and problem text, and build a Gemini prompt embedding the detected emotion and confidence score.

• Story 2: Generate empathetic, field-aware responses via the Gemini API, with a template-based fallback if the API is unavailable or a key is not configured.

• Story 3: Regenerate responses whenever the input text or the "Use AI Response" toggle changes, keeping displayed emotion scores in sync with the response shown.

• Story 4: Maintain session history and CSV logs - every interaction (timestamp, field, problem, emotion, confidence, response, model used) is appended to session state and to CSV for continuous learning.

Epic 5: Streamlit UI Implementation

Builds the interactive interface for submitting problems, viewing predictions, and reviewing analytics.

• Story 1: Responsive layout with session state management for the input form, results, and analytics.

• Story 2: Side-by-side BiLSTM vs BERT comparison view, including mixed-emotion display and per-class confidence bars.

• Story 3: Form controls, quick-example prompts, spinners during analysis, and warnings for empty input.

• Story 4: Analytics dashboard with Plotly - emotion distribution pie chart, emotional-journey line chart, and field/model breakdown bar charts, rendered only once session history exists.

Epic 6: Testing and Deployment Readiness

Validates the complete system end-to-end and confirms readiness for demonstration or deployment.

• Story 1: Validate the UI flow end-to-end - submitting a problem, viewing dual-model predictions, receiving an AI response, and reviewing the analytics dashboard.

• Story 2: Cross-browser verification (consistent layout/widgets across browsers); performance and caching checks (models load only once, stable response times across repeated interactions); environment and final validation (correct environment variables, required models and dependencies present, full workflow functioning); confirmation of system readiness for deployment.

Conclusion:

The **Emotion Detection Learning Support Engine** successfully combines Artificial Intelligence, Natural Language Processing, and Machine Learning to recognize a student's emotional state from text and provide personalized learning support. By integrating BiLSTM and BERT models with the Gemini API, the system identifies emotions such as Bored, Confident, Confused, Curious, and Frustrated - including mixed emotions - and generates empathetic, context-aware guidance to help students work through learning challenges. The Streamlit-based interface offers an interactive, user-friendly experience, while features such as dual-model comparison, session analytics, and interaction logging enhance usability and engagement. Throughout the project, emphasis was placed on accuracy, code correctness (multiple inference-layer bugs were identified and fixed during review), and real-time responsiveness, making the application suitable for educational and emotional-support scenarios. In the future, the system can be further enhanced by supporting a "Neutral"/out-of-domain class, multilingual emotion detection, persistent user accounts for longitudinal progress tracking, and cloud deployment for wider access.